Characterizing Feature Sets for Just-In-Time Defect Prediction: A Systematic Literature Review

Firas Jolha
f.jolha@innopolis.university
Innopolis University
Innopolis, Russia

Adel Azzi
a.azzi@innopolis.university
Innopolis University
Innopolis, Russia

Demid Davletzianov
d.davletzianov@innopolis.university
Innopolis University
Innopolis, Russia

Artem Lisitcin
a.lisitcin@innopolis.university
Innopolis University
Innopolis, Russia

Giancarlo Succi
g.succi@unibo.it
University of Bologna – Alma Mater Studiorum
Bologna, Italy
Constructor Knowledge Labs and Constructor University
Bremen, Germany

Abstract

Just-In-Time (JIT) defect prediction flags software changes that are likely to introduce defects at the moment they are submitted. Every JIT model depends on the feature set used to describe a change. The literature proposes many such features, from change-level metrics about size, diffusion, history and developer experience to semantic representations learned from code and commit messages. These features are named, defined and extracted in different ways, and the evidence on their usefulness is scattered across studies with different datasets, models and settings. Existing secondary studies cover JIT defect prediction as a whole and treat features as one aspect among several. This paper presents a Systematic Literature Review (SLR) focused on the feature sets themselves. We define five research questions on the types and prevalence of feature sets, their properties and extraction methods, their effect on predictive performance, their behavior under concept drift, and their use in within-project, cross-project, online and effort-aware settings. We describe the review protocol, including the search in six digital libraries, twelve exclusion criteria and a twelve-item quality assessment. The search returned 37,566 records, of which 212 primary studies published between 2006 and 2025 were selected for the review. We also describe the data extraction form and the synthesis plan used to answer the research questions.

CCS Concepts

• **Software and its engineering** → **Software defect analysis**; • **Computing methodologies** → *Machine learning*; • **General and reference** → *Surveys and overviews.*

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, City, Country
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2026/MM
https://doi.org/XXXXXXX.XXXXXXX

Keywords

Just-In-Time defect prediction, defect prediction, feature sets, software metrics, concept drift, machine learning, systematic literature review

ACM Reference Format:
Firas Jolha, Adel Azzi, Demid Davletzianov, Artem Lisitcin, and Giancarlo Succi. 2026. Characterizing Feature Sets for Just-In-Time Defect Prediction: A Systematic Literature Review. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 6 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Software defects increase maintenance costs and reduce system reliability [3]. Defects that reach production can cause failures that are hard and expensive to diagnose and fix [25]. Modern software evolves through small, incremental changes. Under continuous integration and delivery, these changes are often integrated several times a day [9, 23]. Quality assurance must keep up with this pace. Traditional defect prediction estimates the defect-proneness of modules, files, or classes at release time [7, 22]. In this setting, its predictions come too late and are too coarse-grained. They also do not tell developers which change to inspect or who should inspect it [15].

Just-In-Time (JIT) defect prediction overcomes these limitations by operating at the level of individual changes (commits). As soon as a change is submitted, the JIT model assesses whether it is likely to be defect-inducing, that is, whether it causes a defect which will later have to be fixed [15, 16, 19]. Identifying defect-inducing changes at this stage has three practical benefits [15]:

- The amount of code to inspect is small.
- The author still remembers the design decisions behind the change.
- It is clear who is responsible for the change.

Whatever learning algorithm it uses, every JIT model relies on how a change is represented, that is, on its feature set. Many different features have been proposed in the literature [6, 10, 13, 15, 21,

29]. Early studies described changes with hand-crafted change-level metrics that captured the size, diffusion, purpose and history of a change and the experience of the developers who made it [15]. Later research added features derived from commit messages, code review activity, static analysis, and the social and process context of development [2, 6]. More recently, deep learning methods learn semantic representations directly from the changed code and the commit message instead of relying on manually engineered metrics [10–12, 27]. Even so, simple change metrics can remain highly competitive with these approaches [28]. As a result, features vary greatly in what they capture, in the artifacts they are extracted from (for example, version control metadata, code diffs, issue trackers, or review systems), and in the tools and assumptions needed to compute them.

This variety leads to three problems. First, there is no shared understanding of which feature sets are used, how they relate to one another, and how they are defined and extracted. The same features often appear under different names, and extraction methods are rarely reported in full, which makes results hard to compare or reproduce. Second, the evidence on which features actually contribute to predictive performance is spread across studies that use different datasets, models, evaluation criteria, and feature combinations. Third, the value of a feature is not constant. As projects evolve, both the distribution of feature values and their association with defect-proneness change over time. This is known as concept drift, and it means that features that are useful at one time may stop being useful later [18].

The usefulness of a feature also depends on the setting in which the model is applied. A model can be used within a single project or across projects [5, 14]. It can be trained once offline or updated online as new changes arrive [4, 17]. It can also be effort-aware. An effort-aware model ranks changes by their predicted defect-proneness per unit of inspection effort, usually measured as the number of modified lines [15, 26]. Each setting places its own demands on the features. In cross-project prediction, features must transfer to projects that were not used for training. In online prediction, defect labels arrive with a delay [4]. In effort-aware prediction, simple unsupervised models built on change metrics can perform as well as supervised ones [26].

Existing secondary studies of JIT defect prediction [1, 29] give an overview of the field, including datasets, learning algorithms, evaluation measures and application scenarios, and treat features as one aspect among several. To the best of our knowledge, no systematic review has focused on the feature sets themselves, that is, on their types and prevalence, their properties and extraction, their contribution to predictive performance, their behavior over time, and their use in practice.

To address this gap, we conduct a Systematic Literature Review (SLR) of the feature sets used in JIT defect prediction, covering peer-reviewed primary studies published over the past two decades. The review is guided by the following research questions:

RQ1 What kinds of feature sets are used in JIT defect prediction, which ones are used most often, and how has their use changed over time?

RQ2 How are these feature sets characterized in terms of their properties and the methods used to extract them?

RQ3 What effect do different feature sets have on the predictive performance of JIT defect prediction models?

RQ4 How does the dynamic behavior of features (concept drift) affect their usefulness in JIT defect prediction?

RQ5 How are the feature sets employed in JIT defect prediction used in practice, and what challenges and limitations arise in different settings?

This paper makes the following contributions:

- A review protocol for studying feature sets in JIT defect prediction, including the search strategy, twelve exclusion criteria, and a twelve-item quality assessment tailored to feature-related reporting.
- The outcome of study selection, in which 212 primary studies were selected from 37,566 initial records.
- A data extraction form and synthesis plan that link each extracted item to the research questions. The planned synthesis covers the classification and prevalence of feature sets (RQ1), their properties, data sources and extraction tools (RQ2), the predictive performance reported for individual feature sets and their combinations, without running new experiments (RQ3), whether primary studies analyze concept drift and how they handle it (RQ4), and the use of feature sets in cross-project and effort-aware prediction together with the challenges reported (RQ5).

The rest of this paper is organized as follows. Section 2 compares existing secondary studies with this review and gives background on JIT defect prediction. Section 3 describes the review protocol, covering the search strategy, the inclusion and exclusion criteria, study selection, quality assessment, and data extraction and synthesis. Section 4 summarizes the current state of the review and the next steps.

2 Background and Related Work

This section first compares existing secondary studies on defect prediction with the present review. It then gives background on JIT defect prediction and on the change metrics most often used in it (Section 2.1).

Several secondary studies have examined software defect prediction. However, most of them focus on the module, file, or class level rather than on individual changes. Hall et al. [7] reviewed studies on software defect prediction and analyzed how modeling approaches affect model performance. Radjenović et al. [22] conducted an SLR on software fault prediction metrics and found that object-oriented and process metrics outperform traditional size and complexity metrics. Pal and Sillitti [20] reviewed the literature on cross-project defect prediction and software metrics, covering data pre-processing and feature selection. Qiu et al. [21] surveyed the features used in software defect prediction and found a shift from traditional software metrics to semantic and network metrics, and from feature selection to deep learning. These reviews cover features in detail, but none focuses on the prediction of individual changes or on the temporal behavior of features.

Only a few secondary studies have focused on JIT defect prediction. Zhao et al. [29] reviewed 67 JIT studies, summarized best practices for each phase of the JIT process, and performed a meta-analysis of the reported performance. They identified data streams

of changes and delayed labels as defining characteristics of the field. Alnagi and Azzeh [1] reviewed 30 JIT studies published between 2012 and 2023 and analyzed the most frequent prediction techniques, datasets and evaluation metrics. Both reviews cover the JIT field as a whole and treat features as one aspect among several. Table 1 compares these secondary studies with this review.

As Table 1 shows, existing secondary studies either analyze features in depth at the module level [21, 22] or cover JIT defect prediction broadly without focusing on its features [1, 29]. Primary studies, in turn, usually propose or evaluate one feature representation within a single setting. As a result, no existing work systematically characterizes the feature sets used for JIT defect prediction. In particular, no review addresses which types of features are used and how this has changed over time (RQ1), how they are defined and extracted (RQ2), how they affect predictive performance (RQ3), how their usefulness changes under concept drift (RQ4), and what limits their use across practical settings (RQ5). To the best of our knowledge, this is the first SLR that characterizes the feature sets used in JIT defect prediction.

2.1 Background on JIT Defect Prediction

JIT defect prediction determines whether a change is likely to introduce a defect at the time it is made [15, 16, 19]. Training data for this task is usually obtained by tracing bug-fixing commits back to the changes that introduced the fixed lines. This procedure is most often carried out with the SZZ algorithm [24]. Each change is then represented by a feature set, and a classifier or ranking model is trained on this labeled history.

The first feature sets consisted of hand-crafted metrics computed from version control data. Mockus and Weiss [19] predicted which changes are likely to be defect-inducing from the size of the changes, their diffusion, and the experience of the developers who made them. Kim et al. [16] used the terms in the modified code and the commit metadata as features. Kamei et al. [15] proposed 14 change-level metrics grouped into five dimensions, including diffusion, size, purpose, history, and developer experience. These metrics, listed in Table 2, are still the most common baseline in the field.

The added value of more detailed representations is still debated. Zeng et al. [28] showed that a simple model using only the number of added lines in a change can match or even beat deep JIT models. Herbold et al. [8] found that only about half of the bug-fixing commits identified by SZZ actually fix bugs. In their study, these label errors were a more severe threat to the validity of defect prediction results than the use of small feature sets. Taken together, these results indicate that how features and labels are defined and extracted can matter as much as the choice of learning algorithm.

3 Methodology

This study follows the SLR approach to identify and analyze empirical research on feature sets used in JIT defect prediction. The review process consists of literature search, application of eligibility criteria, study selection, quality assessment, and data extraction and synthesis.

3.1 Literature Search

We searched six academic databases and digital libraries, namely Google Scholar, SpringerLink, ACM Digital Library, ScienceDirect, Scopus and IEEE Xplore. These sources were chosen to cover publications in software engineering and empirical defect prediction. The search was conducted between September 2025 and October 2026.

The search query combines terms related to defect-inducing changes, software faults and defects, commits and changes, prediction approaches, and feature sets or metrics. The same query was applied to all six sources:

```
(("buggy" OR "bug inducing" OR "anomalous" OR
"just in time defect" OR "just in time bug" OR
"JIT defect" OR "JIT bug") AND ("fault" OR "defect")
AND ("commit" OR "change")) AND
("detection" OR "classification" OR "prediction") AND
("feature set" OR "features" OR "feature selection" OR
"feature engineering" OR "feature importance" OR
"feature characterization" OR "metrics")
```

The initial search identified 37,566 records. Table 3 gives the number of records retrieved from each source.

3.2 Inclusion and Exclusion Criteria

The inclusion and exclusion criteria ensure that the selected studies are relevant to the review topic and provide enough empirical evidence. A study was included if it met all of the following criteria:

- **IC1:** The publication is a peer-reviewed journal article or conference paper.
- **IC2:** The study addresses defect prediction at the commit or change level.
- **IC3:** The study explicitly discusses features, feature sets, or metrics used in JIT defect prediction.
- **IC4:** The publication is written in English and its full text is accessible.
- **IC5:** The publication falls within the selected 20-year publication window.

A study was excluded if any of the following applied:

- **EC1:** The publication is not written in English.
- **EC2:** The publication is not peer-reviewed or does not provide sufficient relevant information.
- **EC3:** The publication is a duplicate or substantially overlaps with another report without providing sufficiently distinct evidence.
- **EC4:** The study is outside software engineering or does not address software defect prediction.
- **EC5:** The study does not report empirical results.
- **EC6:** The study does not include change-level experiments and focuses only on testing, code smells, or traditional module-, file-, or class-level defect prediction.
- **EC7:** The study does not use features to build a JIT defect prediction model.
- **EC8:** The study provides insufficient information about datasets, features, or feature extraction or construction.
- **EC9:** Essential bibliographic information, such as the title, authors, or publication year, is missing.
- **EC10:** The publication is not a primary study, for example, a literature review or systematic literature review.

Table 1: Comparison of this review with related secondary studies on software defect prediction.

Study	Studies (period)	Feature focus	Extraction methods	Impact on performance	Concept drift	Settings
Module-level defect prediction						
Hall et al. [7]	208 (2000–2010)	(✓)	–	(✓)	–	–
Radjenović et al. [22]	106 (1991–2011)	✓	–	✓	–	–
Pal and Sillitti [20]	[n]	(✓)	–	(✓)	–	CP
Qiu et al. [21]	[n]	✓	✓	(✓)	–	[verify]
Change-level (JIT) defect prediction						
Zhao et al. [29]	67	(✓)	(✓)	(✓)	(✓)	CP, OL, EA
Alnagi and Azzeh [1]	30 (2012–2023)	–	–	–	–	CP, EA
This review	**212 (2006–2025)**	✓	✓	✓	✓	**WP, CP, OL, EA**

✓: a central research question of the study; (✓): discussed but not a primary focus; –: not addressed. Settings: WP = within-project, CP = cross-project, OL = online, EA = effort-aware; – = not applicable (not change-level).

Table 2: The 14 change-level metrics proposed by Kamei et al. [15].

Dimension	Metric	Description
Diffusion	NS	Number of modified subsystems
	ND	Number of modified directories
	NF	Number of modified files
	Entropy	Distribution of modified code across files
Size	LA	Lines of code added
	LD	Lines of code deleted
	LT	Lines of code in a file before the change
Purpose	FIX	Whether or not the change is a defect fix
History	NDEV	Number of developers that changed the modified files
	AGE	Average time interval between the last and the current change
	NUC	Number of unique changes to the modified files
Experience	EXP	Developer experience
	REXP	Recent developer experience
	SEXP	Developer experience on a subsystem

Table 3: Records retrieved from each search source

Search source	Records retrieved
Google Scholar	18,901
SpringerLink	10,065
ACM Digital Library	3,062
ScienceDirect	478
Scopus	4,999
IEEE Xplore	61
Total	37,566

- **EC11:** The full text is inaccessible.
- **EC12:** The paper scores below the threshold of the quality assessment (Section 3.4).

3.3 Study Selection

Study selection consisted of duplicate removal, title and abstract screening, and full-text eligibility assessment. First, duplicate records were removed from the search results. The remaining records were screened on their titles and abstracts to identify potentially relevant publications. The full texts of the selected publications were then assessed against the inclusion and exclusion criteria.

The screening work was divided among the members of the research team. Selection decisions were recorded in a spreadsheet containing bibliographic information, the source database, and the inclusion or exclusion status of each publication.

The selection process resulted in 212 primary studies, published between 2006 and 2025. Table 4 gives the number of records at each stage, and Figure 1 shows the same process as a PRISMA diagram.

Table 4: Study selection results

Selection stage	Records
Records identified through database searches	37,566
Duplicate records removed	73
Records screened by title and abstract	37,493
Records excluded during title and abstract screening	36,933
Full-text publications assessed for eligibility	560
Full-text publications excluded	348
Studies included in the final review	212

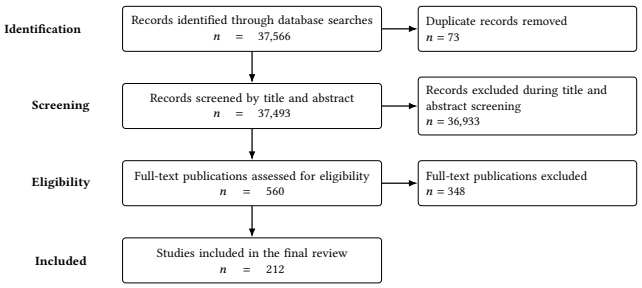


Figure 1: PRISMA diagram of the study selection process

3.4 Quality Assessment

The selected studies were assessed with twelve quality assessment questions (QA1–QA12). The questions characterize the features and

metrics used, how the experimental methods are reported, and how far each study investigates specific prediction settings and challenges. They cover feature characteristics and feature sets, semantic features, dataset descriptions, feature extraction and selection methods, validation procedures, feature combinations, online prediction, concept drift, effort-aware prediction, and cross-project prediction. Each question is answered on a three-point scale, with *Yes* worth 1 point, *Maybe* 0.5 points and *No* 0 points. Table 5 lists the questions and the criteria for each answer.

All twelve questions carry equal weight. The overall quality score of a study is the sum of its scores over QA1–QA12 and ranges from 0 to 12. Studies with an overall score below [3] (25% of the maximum) were excluded under EC12. We chose a low threshold because QA8–QA12 address specialized settings, namely online prediction, concept drift, effort-aware prediction and cross-project prediction, that many otherwise relevant JIT studies do not investigate. The appraisal is therefore meant to remove only studies with minimal methodological and feature-related reporting, not to penalize studies for leaving out optional settings. The assessment results are also used to characterize the included studies in terms of feature-related practices, methodological reporting, and the prediction settings they investigate.

3.5 Data Extraction and Synthesis

A structured data extraction form is used to collect information from the full texts of the included studies. It extends the screening spreadsheet, which already holds the title, authors, publication year, link, search source, and inclusion or exclusion status of each study. The form captures the following items:

- **Feature characteristics:** feature names, descriptions, categories, and calculation formulas (RQ1, RQ2).
- **Feature sets and extraction:** feature sets used, feature extraction methods and their descriptions, extraction tools, links to the tools, and descriptions of their functionality (RQ1, RQ2).
- **Predictive performance and experimental context:** performance metrics, reported effectiveness of feature sets, contribution of features or feature sets to predictive performance where reported, machine learning models, number and names of projects, their programming languages, and project categories (RQ3).
- **Concept drift:** whether concept drift is addressed, methods used to detect it, feature sets reported as robust to it, and differences in model performance associated with it (RQ4).
- **Prediction settings:** model performance in within-project and cross-project settings (RQ5).

The extracted information is synthesized with descriptive summaries and qualitative analysis. Descriptive summaries characterize the included studies, classify feature categories, and identify frequently reported features and temporal patterns. The qualitative synthesis compares feature definitions, extraction methods, experimental settings, predictive performance findings, and reported limitations.

The review synthesizes the predictive performance reported by primary studies and does not run new prediction experiments. Since studies differ in datasets, evaluation protocols and performance measures, their findings are interpreted in the context of their original experimental settings. Direct numerical comparisons are made only when they are methodologically justified. The synthesis also separates topics that primary studies explicitly investigate from those for which they report no evidence.

4 Current Status and Next Steps

This paper presented the protocol of an SLR on the feature sets used in JIT defect prediction and the outcome of its study selection. The search in six digital libraries returned 37,566 records, and 212 primary studies published between 2006 and 2025 were selected for the review. The next step is to extract data from the selected studies with the form described in Section 3.5 and to synthesize it to answer RQ1–RQ5. The expected outcome is a classification of the feature sets used in JIT defect prediction, a description of how they are extracted, a summary of the evidence on their predictive value, and an account of how primary studies deal with concept drift and with cross-project, online and effort-aware settings.

References

[1] Eman Alnagi and Mohammad Azzeh. 2024. Just-in-Time Software Defect Prediction Techniques: A Survey. In *Proceedings of the 15th International Conference on Information and Communication Systems (ICICS)*. 1–6. doi:10.1109/ICICS63486.2024.10638276

[2] Amir T. Barzoki and Kostas Kontogiannis. 2025. Predicting Bug Inducing Commits Using Commit-State Transition Analysis. In *Proceedings of the 49th IEEE Annual Computers, Software, and Applications Conference (COMPSAC)*. doi:10.1109/COMPSAC65507.2025.00165

[3] Barry Boehm and Victor R. Basili. 2001. Software Defect Reduction Top 10 List. *Computer* 34, 1 (2001), 135–137. doi:10.1109/2.962984

[4] George G. Cabral, Leandro L. Minku, Emad Shihab, and Suhaib Mujahid. 2019. Class Imbalance Evolution and Verification Latency in Just-in-Time Software Defect Prediction. In *Proceedings of the 41st IEEE/ACM International Conference on Software Engineering (ICSE)*. 666–676. doi:10.1109/ICSE.2019.00076

[5] Hongming Dai, Jianqing Xi, and Hong-Liang Dai. 2025. Improve Cross-Project Just-in-Time Defect Prediction with Dynamic Transfer Learning. *Journal of Systems and Software* 219 (2025), 112214. doi:10.1016/j.jss.2024.112214

[6] Emanuela Guglielmi, Andrea D'Aguanno, Rocco Oliveto, and Simone Scalabrino. 2025. Enhancing Just-In-Time Defect Prediction Models with Developer-Centric Features. In *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. 52–62. doi:10.1109/MSR66628.2025.00021

[7] Tracy Hall, Sarah Beecham, David Bowes, David Gray, and Steve Counsell. 2012. A Systematic Literature Review on Fault Prediction Performance in Software Engineering. *IEEE Transactions on Software Engineering* 38, 6 (2012), 1276–1304. doi:10.1109/TSE.2011.103

[8] Steffen Herbold, Alexander Trautsch, Fabian Trautsch, and Benjamin Ledel. 2022. Problems with SZZ and Features: An Empirical Study of the State of Practice of Defect Prediction Data Collection. *Empirical Software Engineering* 27, 2 (2022), 42. doi:10.1007/s10664-021-10092-4

[9] Michael Hilton, Timothy Tunnell, Kai Huang, Darko Marinov, and Danny Dig. 2016. Usage, Costs, and Benefits of Continuous Integration in Open-Source Projects. In *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 426–437. doi:10.1145/2970276.2970358

[10] Thong Hoang, Hoa Khanh Dam, Yasutaka Kamei, David Lo, and Naoyasu Ubayashi. 2019. DeepJIT: An End-to-End Deep Learning Framework for Just-in-Time Defect Prediction. In *Proceedings of the 16th IEEE/ACM International Conference on Mining Software Repositories (MSR)*. 34–45. doi:10.1109/MSR.2019.00016

[11] Thong Hoang, Hong Jin Kang, David Lo, and Julia Lawall. 2020. CC2Vec: Distributed Representations of Code Changes. In *Proceedings of the 42nd ACM/IEEE International Conference on Software Engineering (ICSE)*. 518–529. doi:10.1145/3377811.3380361

[12] Yuze Jiang, Beijun Shen, and Xiaodong Gu. 2025. Just-in-Time Software Defect Prediction via Bi-modal Change Representation Learning. *Journal of Systems and Software* 219 (2025), 112253. doi:10.1016/j.jss.2024.112253

[13] Xiaolin Ju, Yi Cao, Xiang Chen, Lina Gong, Vaskar Chakma, and Xin Zhou. 2025. JIT-CF: Integrating Contrastive Learning with Feature Fusion for Enhanced Just-In-Time Defect Prediction. *Information and Software Technology* 182 (2025), 107706. doi:10.1016/j.infsof.2025.107706

[14] Yasutaka Kamei, Takafumi Fukushima, Shane McIntosh, Kazuhiro Yamashita, Naoyasu Ubayashi, and Ahmed E. Hassan. 2016. Studying Just-in-Time Defect

Table 5: Quality assessment questions and scoring criteria

ID	Question	Yes (1)	Maybe (0.5)	No (0)
QA1	Does the study use features or metrics that leverage various characteristics of code changes for JIT defect prediction?	Uses features that capture at least two types of code-change characteristics (e.g., change size and diffusion).	Uses features that capture only one type of code-change characteristics (e.g., change size).	Uses no features that capture code-change characteristics (e.g., only developer-related features).
QA2	Does the study identify different sets of features used in JIT defect prediction?	Uses at least two different feature sets (e.g., AST-based features and the expert features of Kamei et al.).	Relies on a single feature set (e.g., the expert features of Kamei et al.).	Does not specify any feature set.
QA3	Does the study use semantic features that represent the underlying semantics of code changes?	Uses both expert and semantic features.	Uses only semantic features.	Uses only expert features.
QA4	Does the study sufficiently describe the datasets used in the experiments?	Provides sufficient details about the datasets used.	Uses some datasets without enough detail about their description.	Lacks a description of the datasets used.
QA5	Does the study report the methods and tools used for feature extraction and selection?	Clearly defines the methods and tools used for extracting and selecting features.	Uses external tools for extracting features.	Lacks the methodology and tools for extracting and selecting features.
QA6	Does the study validate the methods or tools used for feature extraction and selection?	Empirically validates the methods and tools used for feature extraction and selection.	Provides a validation scheme without any empirical work.	Does not validate the methods and tools used for feature extraction and selection.
QA7	Does the study evaluate the impact of different feature combinations on JIT defect prediction?	Examines the effect of different feature sets on JIT defect prediction performance.	Measures the effect of a single feature set on JIT defect prediction performance.	Reports performance for one feature set without characterizing any feature subsets.
QA8	Does the study investigate online JIT defect prediction?	Reports the performance of JIT defect prediction in an online setting.	Discusses online JIT defect prediction without relevant experiments.	Experiments only with a static set of features and does not discuss online prediction.
QA9	Does the study incorporate methods for detecting concept drift in online JIT defect prediction?	Examines several methods for detecting concept drift.	Uses a single method for detecting concept drift.	Does not discuss concept drift.
QA10	Does the study measure the performance of JIT defect prediction models in the presence of concept drift?	Reports changes in model performance in the presence of concept drift.	Provides some information on performance changes under concept drift, but not based on empirical work.	Provides no information on performance changes under concept drift.
QA11	Does the study investigate effort-aware JIT defect prediction?	Uses metrics to measure the performance of effort-aware JIT defect prediction.	Discusses such metrics but does not use them.	Does not mention metrics for effort-aware JIT defect prediction.
QA12	Does the study conduct experiments on cross-project JIT defect prediction?	Provides empirical results for cross-project JIT defect prediction.	Discusses cross-project performance without empirical investigation.	Does not discuss cross-project JIT defect prediction.

Prediction Using Cross-Project Models. *Empirical Software Engineering* 21, 5 (2016), 2072–2106. doi:10.1007/s10664-015-9400-x

[15] Yasutaka Kamei, Emad Shihab, Bram Adams, Ahmed E. Hassan, Audris Mockus, Anand Sinha, and Naoyasu Ubayashi. 2013. A Large-Scale Empirical Study of Just-in-Time Quality Assurance. *IEEE Transactions on Software Engineering* 39, 6 (2013), 757–773. doi:10.1109/TSE.2012.70

[16] Sunghun Kim, E. James Whitehead, Jr., and Yi Zhang. 2008. Classifying Software Changes: Clean or Buggy? *IEEE Transactions on Software Engineering* 34, 2 (2008), 181–196. doi:10.1109/TSE.2007.70773

[17] Xutong Liu, Yufei Zhou, Yutian Tang, Junyan Qian, and Yuming Zhou. 2025. Human-in-the-loop Online Just-in-time Software Defect Prediction: What Have We Achieved and What Do We Still Miss? *Science of Computer Programming* 244 (2025), 103296. doi:10.1016/j.scico.2025.103296

[18] Shane McIntosh and Yasutaka Kamei. 2018. Are Fix-Inducing Changes a Moving Target? A Longitudinal Case Study of Just-in-Time Defect Prediction. *IEEE Transactions on Software Engineering* 44, 5 (2018), 412–428. doi:10.1109/TSE.2017.2693980

[19] Audris Mockus and David M. Weiss. 2000. Predicting Risk of Software Changes. *Bell Labs Technical Journal* 5, 2 (2000), 169–180. doi:10.1002/bltj.2229

[20] Sourabh Pal and Alberto Sillitti. 2022. Cross-Project Defect Prediction: A Literature Review. *IEEE Access* 10 (2022), 118697–118717. doi:10.1109/ACCESS.2022.3221184

[21] Shaoming Qiu, Bicong E, Jingjie He, and Liangyu Liu. 2025. Survey of Software Defect Prediction Features. *Neural Computing and Applications* 37, 4 (2025), 2113–2144. doi:10.1007/s00521-024-10937-1

[22] Danijel Radjenović, Marjan Heričko, Richard Torkar, and Aleš Živkovič. 2013. Software Fault Prediction Metrics: A Systematic Literature Review. *Information and Software Technology* 55, 8 (2013), 1397–1418. doi:10.1016/j.infsof.2013.02.009

[23] Mojtaba Shahin, Muhammad Ali Babar, and Liming Zhu. 2017. Continuous Integration, Delivery and Deployment: A Systematic Review on Approaches, Tools, Challenges and Practices. *IEEE Access* 5 (2017), 3909–3943. doi:10.1109/ACCESS.2017.2685629

[24] Jacek Śliwerski, Thomas Zimmermann, and Andreas Zeller. 2005. When Do Changes Induce Fixes?. In *Proceedings of the 2005 International Workshop on Mining Software Repositories (MSR)*. 1–5. doi:10.1145/1083142.1083147

[25] Gregory Tassey. 2002. *The Economic Impacts of Inadequate Infrastructure for Software Testing*. Planning Report 02-3. National Institute of Standards and Technology.

[26] Yibiao Yang, Yuming Zhou, Jinping Liu, Yangyang Zhao, Hongmin Lu, Lei Xu, Baowen Xu, and Hareton Leung. 2016. Effort-Aware Just-in-Time Defect Prediction: Simple Unsupervised Models Could Be Better than Supervised Models. In *Proceedings of the 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE)*. 157–168. doi:10.1145/2950290.2950353

[27] Yujie Ye, Huiqun Yu, Guisheng Fan, Yuguo Liang, Jianan Dong, and Wentao Chen. 2025. JIT-Align: A Semantic Alignment-Based Ranking Framework for Just-In-Time Defect Prediction. In *Proceedings of the 49th IEEE Annual Computers, Software, and Applications Conference (COMPSAC)*. 1328–1337. doi:10.1109/COMPSAC65507.2025.00166

[28] Zhengran Zeng, Yuqun Zhang, Haotian Zhang, and Lingming Zhang. 2021. Deep Just-in-Time Defect Prediction: How Far Are We?. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. 427–438. doi:10.1145/3460319.3464819

[29] Yunhua Zhao, Kostadin Damevski, and Hui Chen. 2023. A Systematic Survey of Just-in-Time Software Defect Prediction. *Comput. Surveys* 55, 10 (2023), 1–35. doi:10.1145/3567550